ST4237 Midterm 2025 – Solutions

1. Estimate the integral
$$\int_{x=0}^{3}\int_{y=0}^{3}\int_{z=0}^{3} e^{\sin(xy+yz^2)}\,dx\,dy\,dz$$
using the Monte Carlo method.
[at least one correct decimal expected]

   Solution: 38.3

```
import numpy as np
np.random.seed(42)
N = 1_000_000

# Key idea: for any function g and domain [a,b]^d,
# integral = volume * E[g(U)] where U is uniform on [a,b]^d

# Step 1: sample uniformly on [0,3]^3
x = np.random.uniform(0, 3, N)
y = np.random.uniform(0, 3, N)
z = np.random.uniform(0, 3, N)

# Step 2: evaluate the integrand at each sample
f = np.exp(np.sin(x * y + y * z**2))

# Step 3: multiply mean by volume of [0,3]^3
volume = 3**3  # = 27
estimate = volume * np.mean(f)
print(f"Answer: {estimate:.1f}")  # 38.3
```

2. Consider three independent random variables $X, Y, Z \sim \mathcal{N}(0,1)$. Estimate the probability
$$\mathbb{P}\big(\,|X/Y|^2 > Z^2\,\big).$$

   [at least one correct decimal expected]

   Solution: 0.627

```python
# Step 1: simulate N independent copies of (X,Y,Z)
X = np.random.normal(0, 1, N)
Y = np.random.normal(0, 1, N)
Z = np.random.normal(0, 1, N)

# Step 2: check the event |X/Y|^2 > Z^2
# i.e. X^2/Y^2 > Z^2 for each sample
event = (X**2 / Y**2 > Z**2)

# Step 3: P(event) ~ proportion of True
estimate = np.mean(event)
print(f"Answer: {estimate:.3f}")  # 0.628
```

3. Estimate the volume of

$$V = \{(x, y, z) \in \mathbb{R}^3 \,:\, 0 < x < y < z < 2\}.$$

[at least one correct decimal expected]

Solution: $2^3/6 = 1.33$

```python
# Key idea: V is a subset of [0,2]^3.
# Volume(V) = Volume([0,2]^3) * P(x < y < z)
# where (x,y,z) is uniform on [0,2]^3.

# Step 1: sample uniformly on [0,2]^3
x = np.random.uniform(0, 2, N)
y = np.random.uniform(0, 2, N)
z = np.random.uniform(0, 2, N)

# Step 2: check the ordering constraint
ordered = (x < y) & (y < z)

# Step 3: volume = 8 * proportion satisfying x<y<z
vol = 2**3  # = 8
estimate = vol * np.mean(ordered)
print(f"Answer: {estimate:.2f}")  # 1.33
```

4. On average, how many times does one need to roll a fair die until the sum of the rolls strictly exceeds 20.
[at least one correct decimal expected]

Remark: the command

```
np.random.randint(low=a, high=b)
```

generates an integer uniformly at random in the range $\{a, a+1, \ldots, b-1\}$.
[at least one correct decimal expected]

Solution: 6.47

```
# Simulate one experiment: keep rolling until sum > 20
def rolls_until_exceeds_20():
    total = 0
    count = 0
    while total <= 20:
        total += np.random.randint(1, 7)  # 1,...,6
        count += 1
    return count

# Repeat the experiment many times
N_sim = 500_000
results = [rolls_until_exceeds_20()
           for _ in range(N_sim)]

# The answer is the average number of rolls
print(f"Answer: {np.mean(results):.2f}")  # 6.47
```

5. Consider the probability distribution on $[0, 3]$ with density $f(x) \propto \log(1 + x^2)$. Estimate the mean of this distribution using the Monte Carlo method.
[at least one correct decimal expected]

Solution: 2.06

```
# SNIS (Self-Normalized Importance Sampling)
# Target: f(x) proportional to log(1+x^2) on [0,3]
# Proposal: q = Uniform(0,3), so q(x) = 1/3
# Unnormalized weight: w(x) = f_tilde(x)/q(x) = 3*log(1+x^2)
# Since SNIS normalizes, the constant 3 cancels.

x = np.random.uniform(0, 3, N)
w_unnormalized = np.log(1 + x**2)
w_normalized = w_unnormalized / np.sum(w_unnormalized)

# SNIS estimate: E[h(X)] ~ sum( h(x_i) * w_normalized_i )
mu = np.sum(x * w_normalized)
print(f"Answer: {mu:.2f}")  # 2.06
```

6. Consider the probability distribution on $[0, 3]$ with density $f(x) \propto \log(1 + x^2)$. Estimate the standard deviation of this distribution using the Monte Carlo method.
[at least one correct decimal expected]

Solution: 0.652

```python
# SNIS (Self-Normalized Importance Sampling), same setup as Q5
x = np.random.uniform(0, 3, N)
w_unnormalized = np.log(1 + x**2)
w_normalized = w_unnormalized / np.sum(w_unnormalized)

# SNIS estimates of E[X] and E[X^2]
E_X = np.sum(x * w_normalized)
E_X2 = np.sum(x**2 * w_normalized)

# Var(X) = E[X^2] - (E[X])^2, then std = sqrt(Var)
std = np.sqrt(E_X2 - E_X**2)
print(f"Answer: {std:.3f}")  # 0.652
```

7. Consider the density on $[0, 2]$ with density $f(x) = x^3/Z$ where Z is the normalizing constant. One knows from the lectures that one can simulate from this distribution by setting $X = \Phi(U)$ where $U \sim \mathcal{U}(0, 1)$ and Φ is the inverse of the cumulative distribution function of f. Estimate

$$\mathrm{E}[\Phi(U)\,\Phi(1-U)]$$

using the Monte Carlo method.
[at least one correct decimal expected]

Solution: CDF of f is $F(x) = x^4/16$ and inverse CDF is $\Phi(x) = 2\,x^{1/4}$. The expectation is 2.47.

```python
# Z = integral of x^3 from 0 to 2 = 16/4 = 4
# CDF: F(x) = x^4 / 16
# Inverse CDF: Phi(u) = 2 * u^{1/4}
def Phi(u):
    return 2 * u**(1/4)

U = np.random.uniform(0, 1, N)
estimate = np.mean(Phi(U) * Phi(1 - U))
print(f"Answer: {estimate:.2f}")  # 2.47
```

8. Consider the density on $[0, 2]$ with density $f(x) = x^3/Z$ where Z is the normalizing constant. What is the probability that $X > 1.8$ if X is distributed according to f?
[at least one correct decimal expected]

Solution: No simulation needed. From Q7, the CDF is $F(x) = x^4/16$. Therefore

$$\mathrm{P}(X > 1.8) = 1 - F(1.8) = 1 - \frac{1.8^4}{16} = 1 - \frac{10.4976}{16} \approx 0.344.$$

9. Estimate the integral

$$\int_{x=-\infty}^{\infty} \log(1+x^2)\, e^{-x^2/2}\, dx$$

using the Monte Carlo method.
[at least one correct decimal expected]

Solution: 1.336

```
# Key idea: recognize e^{-x^2/2} in the integrand.
# The N(0,1) density is phi(x) = e^{-x^2/2} / sqrt(2*pi).
# So e^{-x^2/2} = sqrt(2*pi) * phi(x), and
# integral = sqrt(2*pi) * E[log(1+X^2)], X ~ N(0,1).

X = np.random.normal(0, 1, N)

# Evaluate the function inside the expectation
g = np.log(1 + X**2)

# Multiply by sqrt(2*pi) to account for the density
estimate = np.sqrt(2 * np.pi) * np.mean(g)
print(f"Answer: {estimate:.3f}")  # 1.336
```

10. Estimate the integral

$$\int_{x=-\infty}^{\infty} \int_{y=-\infty}^{\infty} \frac{\sin(3x^2+2y^2)}{(1+x^2)(1+y^2)}\, dx\, dy$$

using the Monte Carlo method.
[at least one correct decimal expected]

Remark: the Cauchy distribution has density $f(x) = \frac{1}{\pi\,(1+x^2)}$ and can be generated with `np.random.standard_cauchy()`.
[at least one correct decimal expected]

Solution: 1.07

```
# Key idea: the Cauchy density is f(x) = 1/(pi*(1+x^2)),
# so 1/(1+x^2) = pi * f_Cauchy(x).
# The integral becomes:
# int int sin(...) * pi*f(x) * pi*f(y) dx dy
# = pi^2 * E[sin(3X^2 + 2Y^2)], X,Y ~ Cauchy.

X = np.random.standard_cauchy(N)
Y = np.random.standard_cauchy(N)
g = np.sin(3 * X**2 + 2 * Y**2)
estimate = np.pi**2 * np.mean(g)
print(f"Answer: {estimate:.2f}")  # 1.07
```

11. To complete a project, ten tasks $T_1, \ldots, T_{10}$ must be completed. The time to complete each task is exponentially distributed with mean 1 hour. The tasks are independent and are all started at the same time and completed concurrently. Estimate the probability that the project is completed in more than 4 hours.

    Remark: the exponential distribution with mean $\mu > 0$ has density $f(x) = \frac{1}{\mu}\exp(-x/\mu)$ and can be generated with `np.random.exponential(scale=mu)`.
    [at least one correct decimal expected]

    Solution: 0.168

```
# The project finishes when ALL tasks are done,
# so completion time = max of the 10 task times.

# Step 1: simulate 10 Exp(1) task times, N times
tasks = np.random.exponential(1, size=(N, 10))

# Step 2: completion time = max across the 10 tasks
completion_time = np.max(tasks, axis=1)

# Step 3: estimate P(completion_time > 4)
estimate = np.mean(completion_time > 4)
print(f"Answer: {estimate:.3f}")  # 0.168
```

12. Consider two independent random variables $X, Y \sim \mathcal{N}(0, 1)$. Compute:

$$\mathbb{E}[\,XY\,].$$

    [at least one correct decimal expected]

    Solution: No simulation needed. Since X and Y are independent, $\mathrm{E}[XY] = \mathrm{E}[X] \cdot \mathrm{E}[Y] = 0 \times 0 = 0$.

13. Consider two independent random variables $X, Y \sim \mathcal{N}(0, 1)$. Conditioned on the event that $X + Y > 1$, compute the expectation of their product:

$$\mathbb{E}[\,XY \,|X + Y > 1\,].$$

    [at least one correct decimal expected]

    Solution: 0.458

```
# Key idea: to estimate E[g(X,Y) | event],
# simulate many (X,Y), discard those where the
# event does not hold, average g over the rest.

X = np.random.normal(0, 1, N)
Y = np.random.normal(0, 1, N)

# Keep only samples satisfying the condition
mask = (X + Y) > 1

# Average X*Y over the accepted samples
estimate = np.mean(X[mask] * Y[mask])
print(f"Answer: {estimate:.3f}")  # 0.458
```

14. Consider two independent random variables $X, Y \sim \mathcal{N}(0,1)$. We would like to estimate accurately the very rare event that $X^2 + Y^2 > 40$. For this purpose, use importance sampling with proposal distributions

$$X_{IS} \sim \mathcal{N}(0, \sigma_{is}^2 = 5^2) \quad \text{and} \quad Y_{IS} \sim \mathcal{N}(0, \sigma_{is}^2 = 5^2).$$

Consider the quantity

$$\alpha = 10^{10} \times \mathbb{P}\big(X^2 + Y^2 > 40\big).$$

Estimate α using the Monte Carlo method.

Remark: the normal distribution with mean μ and variance σ^2 has density $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ and can be generated with

`np.random.normal(loc=mu, scale=sigma)`.

[at least one correct decimal expected]

Solution: $\alpha \approx 20.6$

```
# Importance Sampling with SNIS
# Target: p(x) = N(0,1), we want E_p[h(X,Y)]
# Proposal: q(x) = N(0, sigma_is^2), wider to capture rare events
# Goal: estimate P(X^2 + Y^2 > 40)

# Step 1: sample from proposal q
sigma_is = 5
X = np.random.normal(0, sigma_is, N)
Y = np.random.normal(0, sigma_is, N)

# Step 2: unnormalized importance weights w = p(x,y) / q(x,y)
# For one coordinate:
#   p(x)/q(x) = sigma * exp(-x^2/2 + x^2/(2*sigma^2))
```

```
# We work in log-space for numerical stability.
log_w_x = np.log(sigma_is) \
        + (-X**2/2 + X**2/(2 * sigma_is**2))
log_w_y = np.log(sigma_is) \
        + (-Y**2/2 + Y**2/(2 * sigma_is**2))
# X,Y independent: w(x,y) = w(x) * w(y)
log_w = log_w_x + log_w_y
w_unnormalized = np.exp(log_w)

# Step 3: normalize weights (SNIS)
w_normalized = w_unnormalized / np.sum(w_unnormalized)

# Step 4: SNIS estimate of P(X^2 + Y^2 > 40)
h = (X**2 + Y**2 > 40).astype(float)
prob = np.sum(h * w_normalized)

alpha = 1e10 * prob
print(f"Answer: alpha = {alpha:.1f}")  # 20.6
```

15. One can approximately simulate from a $\mathcal{N}(0,1)$ distribution by summing 12 independent samples from a $\mathrm{Uniform}([-\frac{1}{2},\frac{1}{2}])$ distribution (i.e. uniform on the interval $[-\frac{1}{2},\frac{1}{2}]$). Estimate the quantity

$$\mathbb{P}\left(\sum_{i=1}^{12} U_i > 0.5\right)$$

where the random variables $U_i \sim \mathrm{Uniform}(-\frac{1}{2},\frac{1}{2})$ are independent.
[at least one correct decimal expected]

Solution: 0.31

```
# Step 1: for each trial, draw 12 Uniform(-1/2,1/2)
U = np.random.uniform(-0.5, 0.5, (N, 12))

# Step 2: compute the sum for each trial
S = np.sum(U, axis=1)

# Step 3: estimate P(S > 0.5)
estimate = np.mean(S > 0.5)
print(f"Answer: {estimate:.2f}")  # 0.31
```

16. Estimate the quantity

$$\alpha = 10^{10} \times \mathbb{P}\left(\sum_{i=1}^{12} U_i > 6\right)$$

where the random variables $U_i \sim \mathrm{Uniform}(-\frac{1}{2},\frac{1}{2})$ are independent.
[at least one correct decimal expected]

Solution: No simulation needed. Each $U_i \in [-\frac{1}{2}, \frac{1}{2}]$, so $\sum_{i=1}^{12} U_i \leq 12 \times \frac{1}{2} = 6$. The sum can never *strictly* exceed 6 (and equals 6 with probability zero for continuous random variables). Therefore $\alpha = 0$.

17. Consider the distribution on $\{-2 \leq x, y, z \leq 2\}$ with density

$$f(x, y, z) \propto (x^2 + y^2 + z^2).$$

Use the rejection sampling method to generate samples (X, Y, Z) from this distribution and estimate the quantity

$$\mathbb{E}[\, X^2 + Y^2 + Z^2 \,].$$

[at least one correct decimal expected]

Solution: 5.06

```
# Rejection sampling with per-sample accept/reject loop.
# Proposal: Uniform([-2,2]^3)
# Target: f(x,y,z) proportional to x^2+y^2+z^2
# M = max of x^2+y^2+z^2 on [-2,2]^3 = 4+4+4 = 12
# Accept with probability (x^2+y^2+z^2) / M

def sample_rejection():
    while True:
        x = np.random.uniform(-2, 2)
        y = np.random.uniform(-2, 2)
        z = np.random.uniform(-2, 2)
        if np.random.uniform() < (x**2+y**2+z**2) / 12:
            return x, y, z

N_rej = 10_000
samples = np.array([sample_rejection()
                    for _ in range(N_rej)])
estimate = np.mean(samples[:,0]**2
                 + samples[:,1]**2
                 + samples[:,2]**2)
print(f"Answer: {estimate:.2f}")  # 5.06
```

18. A random walk $(X_0, X_1, \ldots, X_{50})$ is defined by $X_0 = 0$ and $X_{t+1} = X_t + Z_t$ where $Z_1, Z_2, \ldots$ are i.i.d. random variables with $\mathbb{P}(Z_t = 1) = \mathbb{P}(Z_t = -1) = 1/2$. We define L has the number of times the random walk is at the level zero,

$$L = \sum_{t=0}^{50} \mathbb{I}_{\{X_t = 0\}}.$$

Estimate the mean of L using the Monte Carlo method.
[at least one correct decimal expected]

Solution: 5.73

```
# Step 1: simulate N random walks of 50 steps
# Each step is +1 or -1 with equal probability
steps = 2 * np.random.randint(0, 2, (N, 50)) - 1

# Step 2: positions X_1,...,X_50 via cumulative sum
positions = np.cumsum(steps, axis=1)

# Step 3: count visits to zero
# X_0 = 0 always counts, so L = 1 + #{t>=1: X_t=0}
L = 1 + np.sum(positions == 0, axis=1)
print(f"Mean of L: {np.mean(L):.2f}")  # 5.73
```

19. A random walk $(X_0, X_1, \ldots, X_{50})$ is defined by $X_0 = 0$ and $X_{t+1} = X_t + Z_t$ where $Z_1, Z_2, \ldots$ are i.i.d. random variables with $\mathbb{P}(Z_t = 1) = \mathbb{P}(Z_t = -1) = 1/2$. We define L has the number of times the random walk is at the level zero,

$$L = \sum_{t=0}^{50} \mathbb{I}_{\{X_t=0\}}.$$

Estimate the standard deviation of L using the Monte Carlo method.
[at least one correct decimal expected]

Solution: 3.65

```
# Same simulation as Q18
steps = 2 * np.random.randint(0, 2, (N, 50)) - 1
positions = np.cumsum(steps, axis=1)
L = 1 + np.sum(positions == 0, axis=1)

# Now compute std instead of mean
print(f"Std of L: {np.std(L):.2f}")  # 3.65
```

20. consider 5 points uniformly distributed on the circumference of a circle. Estimate the probability that the 5 points are all contained within a half-circle (i.e. they are all on the same side of a diameter).
[at least one correct decimal expected]

Solution: $5/2^4 = 0.3125$

```
# Key idea: 5 points on a circle lie in a half-circle
# iff the largest arc gap between consecutive
# points is >= pi (i.e. >= half the circle).

# Step 1: place 5 points uniformly on [0, 2*pi)
angles = np.random.uniform(0, 2*np.pi, (N, 5))
```

```python
a = np.sort(angles, axis=1)

# Step 2: compute gaps between consecutive points
gaps = np.diff(a, axis=1)  # 4 interior gaps
# Wraparound gap: from last point back to first
wrap_gap = 2*np.pi - a[:,-1] + a[:,0]

# Step 3: the largest gap among all 5 gaps
max_interior_gap = np.max(gaps, axis=1)
max_gap = np.maximum(max_interior_gap, wrap_gap)

# Step 4: all in a half-circle iff max_gap >= pi
estimate = np.mean(max_gap >= np.pi)
print(f"Answer: {estimate:.4f}")  # 0.3125
```